

기초빅데이터프로그래밍 기말 프로젝트:

학사정보 입력 시스템 설계

담당 교수 : 서경희 교수님
이름 : 이석희
학번 : 20200595

1. 프로젝트 개요

본 프로젝트는 Python 기반 객체지향 프로그래밍을 활용하여 대학 학사정보시스템의 핵심 기능을 구현한 CLI 애플리케이션입니다. 학생, 교수, 관리자 세 사용자 유형에 따라 기능을 분리하고, 수강신청, 성적입력 및 조회, 정보 출력 등의 기능을 제공합니다.

JSON 파일(academic_data.json)을 통해 학생 300 명, 교수 40 명, 과목 239 개, 수강신청 5400 건을 관리하며, 시각화 및 정규표현식 기반 검증, 데코레이터, 예외처리 등의 기술을 통합하여 실무적인 학사 시스템의 흐름을 충실히 반영하였습니다.

1.1 주요 목표

- 실제 학사시스템처럼 **수강신청, 성적 입력 및 조회, 수강 과목 확인** 등 기능을 Python 으로 구현
- JSON 기반 데이터 파일(academic_data.json)을 통해 300 명의 학생, 수천 건의 수강 데이터 관리
- 객체지향 설계, 정규표현식, 데코레이터, matplotlib/seaborn 시각화, 예외처리 등의 고급 Python 기능을 통합 적용

1.2 시스템 주요 구성요소

- CLI 기반 인터페이스: 사용자 로그인 후 역할별 메뉴 제공
- 역할별 클래스: User 를 상속받는 Student, Professor, Admin 클래스로 기능 분리
- 과목 및 수강 클래스: Course, Enrollment 클래스는 실제 수업과 수강기록을 표현
- 데이터 영속성: academic_data.json 파일을 통해 저장/불러오기 구현
- 시각화 기능: GPA 추이 그래프, 성적 분포 그래프 등을 matplotlib 및 seaborn 으로 구현

1.3 시스템 흐름 요약

1. 프로그램 시작 시 academic_data.json 로딩
2. 사용자 로그인 → 학생/교수/관리자 분기
3. 각 사용자에게 따라 수강신청, 성적입력, 학생 추가/삭제 등 기능 수행
4. 프로그램 종료 시 자동 저장 수행

1.4 활용 기술 및 라이브러리

- Python 표준 라이브러리: os, json, random, time, logging, functools, typing, re, sys, datetime, collections
- 기술 지원 옵션: from __future__ import annotations (forward reference 지원)
- 외부 라이브러리: numpy, pandas, matplotlib.pyplot, seaborn, tabulate
- 기술 요소:
 - 데코레이터 기반 권한제어 및 실행시간 측정
 - 객체지향 상속, 정적 메서드, 클래스 변수 활용
 - 정규표현식을 통한 입력 유효성 검사

- CLI 인터페이스를 통한 사용자 상호작용

본 프로젝트는 학사 데이터의 흐름과 구조를 모델링하고, 실제 행정 흐름을 코드로 재현한다는 측면에서 학문적, 실용적 가치를 동시에 갖습니다.

2. 전체 시스템 구조 및 클래스 설계

본 시스템은 객체지향 구조에 따라 사용자(User), 학생(Student), 교수(Professor), 관리자(Admin)를 중심으로 구성되어 있으며, 과목(Course), 수강 정보(Enrollment), 데이터 관리(DataManager), 명령줄 인터페이스(CLI) 등의 클래스로 나뉘어 기능을 분담하고 있습니다.

각 클래스는 다음과 같은 책임(Responsibility)을 가집니다.

클래스명	역할 요약	상속 관계	비고
User	공통 사용자 정보(ID, 비밀번호, 전공 등) 관리	base class	상속용 추상 클래스처럼 사용됨
Student	수강신청, 과목 확인, 성적 조회	User 상속	수강이력 및 GPA 시각화 포함
Professor	담당 과목 성적 입력 및 통계 확인	User 상속	수강생 목록, 분포 시각화 포함
Admin	학생 정보 추가/삭제, 통계 조회	User 상속	관리자 전용 CLI 루프 존재
Course	과목 기본 정보 관리	독립 클래스	과목코드, 학점, 담당교수 등 포함
Enrollment	학생-과목 매핑 및 성적 저장	독립 클래스	JSON에 저장되는 핵심 구조
DataManager	JSON 파일 로딩/저장 및 객체 생성	독립 클래스	자동 저장/복원 기능 포함
CLI	전체 실행 흐름 제어, 로그인/메뉴 구현	독립 클래스	사용자 루프, 권한 분기 포함

각 클래스의 세부 속성과 동작 방식은 다음 장에서 메서드 단위로 설명됩니다.

3. 클래스별 상세 설명 및 메서드 동작 방식

3.1 User

- **역할:** 모든 사용자(Student, Professor, Admin)의 공통 속성 및 기능을 정의하는 상위 클래스입니다. 사용자 ID(uid), 이름, 전공, 국적 등의 기본 정보를 포함하고, 인증과 역할 확인 기능을 제공합니다. 이 클래스는 직접 인스턴스화되지 않으며 상속을 통해 기능을 공유합니다.
- **주요 메서드:**
 - `__init__(...)`: 사용자 정보 초기화
 - `authenticate(password: str) -> bool`: 사용자 비밀번호 검증
 - `get_menu() -> List[str]`: 사용자 유형에 따라 CLI 메뉴 리스트 반환

3.2 Student

- **역할:** Student 클래스는 사용자의 역할 중 하나로, 수강신청, 수강 내역 확인, 성적 조회 기능을 수행합니다. 학년, 이수학기, 지도교수 등의 추가 정보를 포함하며, 학기별 GPA 를 계산하고 시각화하는 기능도 담당합니다. 수강신청 시 중복 여부 확인, 과목 정원 확인 및 무작위 대기열 처리 등이 구현되어 있습니다.
- **주요 메서드:**
 - `register_course(course: Course, enrollments: list) -> bool`: 과목 등록을 수행하며, 무작위 대기열 알고리즘을 사용하여 성공 여부를 결정함. 중복 등록 및 정원 초과도 예외 처리함.
 - `drop_course(course_code: str, enrollments: list)`: 이미 수강 중인 과목을 취소하고 Enrollment 객체를 제거함.
 - `check_course(enrollments: list, courses: list)`: 현재 수강 중인 과목 목록을 표 형식(pandas + tabulate)으로 출력함.
 - `check_grade(enrollments: list)`: 수강 학기별 성적을 기반으로 GPA 를 계산하고, matplotlib 을 사용해 학기별 GPA 변화를 시각화함.
 - `@staticmethod gpa_from_grades(grades: list[str]) -> float`: 학점 리스트를 받아 GPA 를 계산하는 정적 메서드. A+, B0 등 성적을 4.5 만점 기준으로 환산.
 - `@classmethod total_students(cls) -> int`: 전체 학생 수 반환 (클래스 변수 기반)

3.3 Professor

- **역할:** Professor 클래스는 교수 역할에 해당하며, 자신이 담당하는 과목의 수강생 목록을 조회하고 성적을 입력하거나 분포 통계를 시각화할 수 있습니다. 사용자 인증 및 메뉴 선택은 상속받은 User 클래스 기능을 사용하고, 고유한 기능만 별도로 정의합니다.
- **주요 메서드:**
 - `assign_grade(course_code: str, student_id: str, grade: str, enrollments: list)`: 담당 과목의 특정 학생에게 성적을 입력하며, 권한이 없는 과목이나 학생에 대해선 예외를 발생시킵니다.

- `list_students(course_code: str, enrollments: list, students: list)`: 특정 과목을 수강하는 학생 목록을 출력하며, 학번, 이름, 전공을 포함한 테이블을 보여줍니다.
- `plot_course_stats(course_code: str, enrollments: list)`: 해당 과목의 성적 분포를 `seaborn` 으로 시각화하며, 빈도 수 기준 막대그래프를 생성합니다.

3.4 Admin

- **역할**: Admin 클래스는 전체 데이터베이스 관리 권한을 가지며, 학생과 교수, 과목 정보를 추가/삭제할 수 있는 기능을 제공합니다. 관리자 모드로 로그인한 경우 CLI 루프는 별도로 분기되며, 이때 `check_time` 데코레이터가 부착된 메서드들이 실행됩니다.
- **주요 메서드**:
 - `add_student(students: list, **kwargs)`: 새로운 학생 객체를 생성하여 리스트에 추가하며, 학번 중복 여부, 전공 유효성 등을 검증합니다.
 - `delete_student(students: list, enrollments: list, uid: str)`: 해당 학번의 학생과 그 학생이 수강한 Enrollment 내역을 모두 제거합니다.
 - `show_counts(students: list, professors: list, courses: list)`: 현재 저장된 학생, 교수, 과목 수를 출력합니다.
 - Admin 의 method 에는 실행시간 측정을 위해 `@check_time` 데코레이터가 적용되어 있습니다.

3.5 Course

- **역할**: Course 클래스는 각 과목의 메타데이터(과목코드, 분반, 담당 교수, 학점, 수강 가능 인원, 시간표 등)를 관리합니다. 수강신청 시 정원 확인에 활용되며, CLI 화면에서 과목 정보 리스트로 출력됩니다.
- **주요 속성**:
 - `code (str)`: 과목 코드
 - `name (str)`: 과목명
 - `major (str)`: 개설 전공
 - `professor (Professor)`: 담당 교수 ID
 - `section (int)`: 분반 번호
 - `credit (int)`: 학점
 - `capacity (int)`: 최대 정원
 - `schedule (str)`: 강의 시간대 및 강의실
 - `year (int), semester (int)`: 개설 학년도 및 학기

3.6 Enrollment

- **역할**: Enrollment 클래스는 학생과 과목 사이의 수강 관계 및 성적 정보를 저장합니다. 실제 JSON 저장 시에도 학번, 과목코드, 학기, 성적만 저장되며, 실행 시 객체 관계로 복원됩니다. 성적이 입력되기 전에는 `None` 값이 할당됩니다.
- **주요 속성**:
 - `student_id (str)`: 수강한 학생의 학번

- `course_code (str)`: 수강한 과목의 코드
- `year (int), semester (int)`: 수강 학년도 및 학기
- `grade (str | None)`: 성적 (예: A+, B0 등)

3.7 DataManager

- **역할**: DataManager 클래스는 `academic_data.json` 파일을 읽어 각 객체(Student, Professor, Course, Enrollment)를 복원하고, 프로그램 종료 시 다시 저장하는 책임을 집니다. 클래스 이름으로 객체를 분기 처리하며 객체 간 관계 매핑을 처리합니다.
- **주요 메서드**:
 - `_load(path: str) -> dict`: JSON 파일을 불러와 각 클래스에 맞게 리스트를 초기화합니다. Enrollment 항목은 `student_id, course_code` 기준으로 객체 매핑 처리합니다.
 - `_save(path: str, students, professors, courses, enrollments)`: 현재 메모리에 있는 객체들을 JSON 직렬화 형식으로 저장합니다.
 - `@staticmethod _user_to_dict(u: User) -> Dict[str, Any]`: User 또는 자식 클래스(Student, Professor 등)의 인스턴스를 JSON 저장용 딕셔너리로 변환합니다.
 - `@staticmethod _course_to_dict(c: Course) -> Dict[str, Any]`: Course 객체는 내부에 복잡한 참조가 없기 때문에 단순히 `'__dict__'`를 복사하여 그대로 직렬화할 수 있습니다. 이 메서드는 JSON 파일에 저장할 때 쓰이며, 복원 시에는 역으로 해당 딕셔너리에서 Course 인스턴스를 재생성합니다.

4. CLI 흐름 구조 및 주요 동작

4.1 로그인 처리

- 관리자 모드 여부 확인 → 일반 사용자 로그인 분기

4.2 사용자별 루프 흐름

- `CLI.run()` → 사용자 타입별 분기:
 - `_admin_loop()`
 - `_student_loop()`
 - `_professor_loop()`

4.3 학생 루프 흐름 예시

- 메뉴: register, drop, courses, grades, info, quit
- register 선택 시 → 개설과목 출력 → 과목코드 입력 → 수강신청 처리

4.4 교수 루프 흐름 예시

- 메뉴: list, grade, stats, info, quit
- stats 선택 시 → 성적 분포 시각화 출력

4.5 관리자 루프 흐름 예시

- 메뉴: 학생 추가, 삭제, 총원 조회
- `add_student()` → CLI 입력으로 전공/지도교수/이수학기 입력

5. 데코레이터 설계 및 활용

본 프로젝트에서는 공통된 기능(로그 기록, 권한 확인, 실행 시간 측정 등)을 각 메서드에 반복 작성하지 않기 위해 커스텀 데코레이터를 설계하였습니다. 이들은 CLI 흐름의 제어, 보안, 성능 측정에 핵심적인 역할을 하며, Python의 `functools.wraps`를 사용해 원래 함수의 메타정보도 보존합니다.

5.1 `check_time`

- **역할:** 함수 실행 시간을 측정하여 화면에 출력합니다.
- **사용 위치:** 관리자(Admin) 클래스의 `show_counts()` 등, 데이터 처리량이 많은 함수에 적용
- **출력 예시:**
`[Time] _admin_show_counts took 0.0008 s`

5.2 `log_access`

- **역할:** 데코레이터가 부착된 함수가 호출될 때마다 호출 시간, 사용자 ID, 함수명을 로그 파일(`access.log`)에 기록합니다.
- **구현 방식:** `datetime.now()`로 시간 스탬프 생성 후 logging 모듈을 통해 파일 기록
- **로그 포맷:**
`2025-06-26 20:54:48,796 [INFO] Called CLI._admin_show_counts`
- **사용 위치:** `check_grade()`, `assign_grade()`, `register_course()` 등 주요 사용자 행동 메서드에 적용

5.3 `require_admin` / `require_professor` / `require_student`

- **역할:** 로그인한 사용자 객체의 클래스 타입을 기준으로 함수 실행 전 권한을 확인합니다. 잘못된 권한의 사용자는 `AuthorizationError`를 통해 접근이 차단됩니다.
- **내부 구현 방식:** 세 데코레이터 모두 공통 구조를 가지며, 내부에서 `isinstance(current_user, RoleClass)` 형태로 검사하며 예외 메시지는 함수명과 함께 출력됩니다.
- **사용 위치:**
 - `@require_admin`: Admin 클래스의 `add_student()`, `delete_student()` 등
 - `@require_professor`: Professor 클래스의 `assign_grade()`, `plot_course_stats()` 등
 - `@require_student`: Student 클래스의 `register_course()`, `check_grade()` 등
- **예외 메시지 예시:**
`AuthorizationError: 이 기능은 관리자만 사용할 수 있습니다.`
- **예시 코드:**

```
def require_role(required_cls_name):
    def decorator(func):
        @wraps(func)
        def wrapper(self, *args, **kwargs):
            user = getattr(self, 'current_user', self)
            required_cls = globals()[required_cls_name]
            if not isinstance(user, required_cls):
                raise AuthorizationError(
                    f"{type(user).__name__} is not allowed to call {func.__name__}"
                )
            return func(self, *args, **kwargs)
        return wrapper
    return decorator
```

`` 함수 실행 전에 현재 로그인한 사용자의 역할을 검사하여, 권한이 없을 경우 `AuthorizationError` 예외를 발생시킵니다.

- 구현 방식: CLI 클래스가 보유한 current_user 객체의 타입을 확인하여 판단
- 예외 메시지 예시:

Error: 이 기능은 교수만 사용할 수 있습니다.

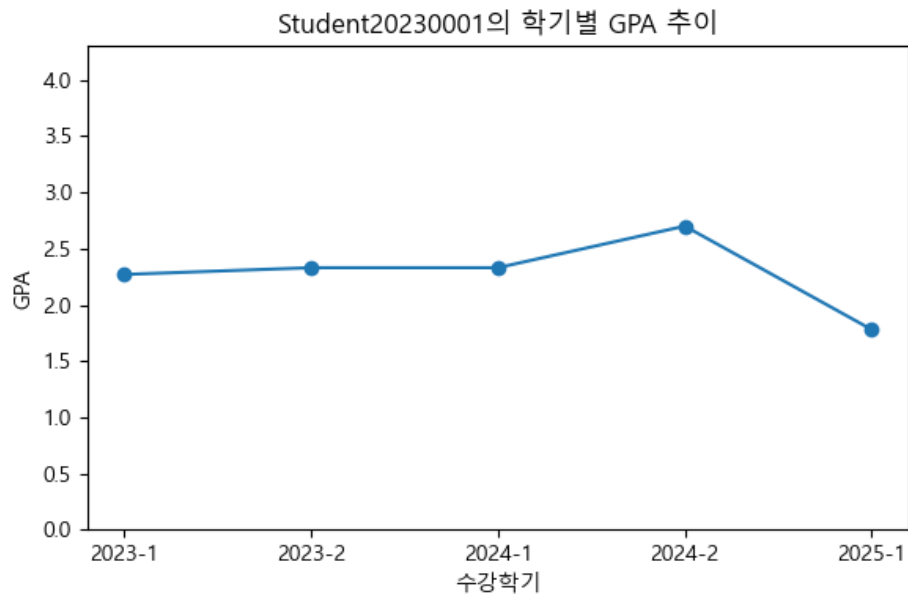
- 사용 위치:
 - require_admin: add_student(), delete_student() 등
 - require_professor: assign_grade(), plot_course_stats() 등
 - require_student: register_course(), check_grade() 등

6. 시각화 및 통계 기능

본 시스템은 시각적 분석을 통해 사용자(학생/교수)가 자신의 학업 성과 및 과목 성적 분포를 명확히 인지할 수 있도록 설계되었습니다. 이를 위해 matplotlib, seaborn, pandas 등의 시각화 라이브러리를 활용하였으며, CLI 환경에서도 그래프 출력이 가능하도록 구성하였습니다.

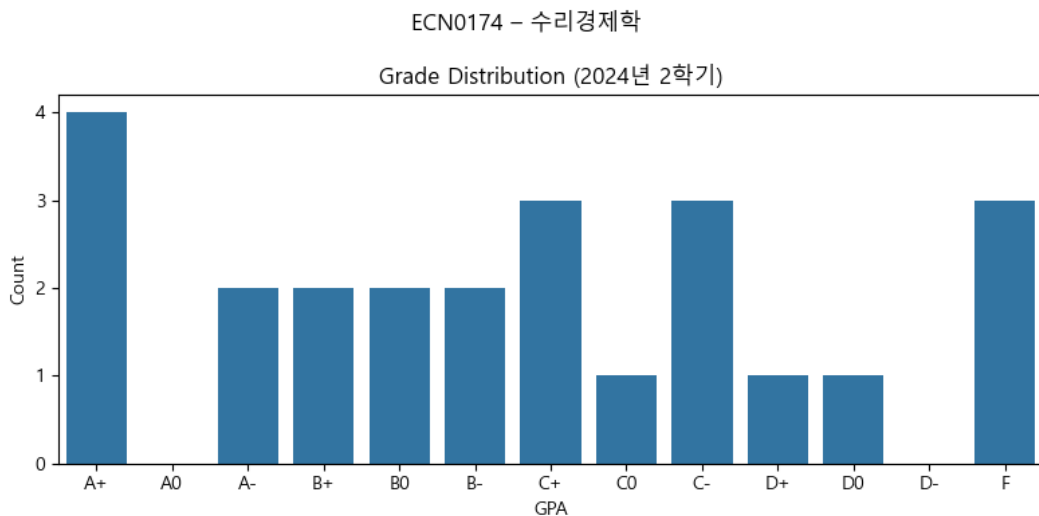
6.1 GPA 시각화 (학생용)

- 역할: 학생은 check_grade() 메서드를 통해 자신이 이수한 학기의 GPA 추이를 확인할 수 있으며, 학기별 GPA 가 선 그래프(line plot)로 시각화됩니다.
- 사용 라이브러리: matplotlib
- 세부 구현:
 - GPA 는 연도 및 학기를 기준으로 정렬된 후 시계열 그래프로 표시됩니다.
 - 시각화 시 plt.plot() 및 plt.xticks(rotation=45) 등을 활용하여 CLI 에서도 읽기 쉬운 형태로 출력됩니다.
- 예시 출력



6.2 성적 분포 시각화 (교수용)

- **역할:** 교수가 특정 과목에 대해 성적 분포를 확인하고, 학습 성과의 편향 여부를 파악하는 데 도움을 줍니다.
- **사용 메서드:** `plot_course_stats()`
- **사용 라이브러리:** `seaborn`, `matplotlib`
- **세부 구현:**
 - 성적이 입력된 학생들의 `grade` 필드를 기준으로 빈도수를 집계하고, `seaborn.countplot()`으로 시각화합니다.
 - A+, A0, A-, B+, ..., F, None 등 다양한 성적 등급이 X 축에 표현되며, Y 축은 학생 수를 나타냅니다.
- **추가 통계:**
 - 평균, 중앙값, 최댓값, 최솟값, 분산 등의 수치도 함께 출력되어 정량적 분석을 지원함
- **예시 출력:**



7. 예외 처리 및 정규표현식 기반 입력 검증

7.1 사용자 정의 예외 클래스

- **AcademicError**: 일반적인 학사 시스템 오류 상황에 사용됩니다. 예: 과목이 존재하지 않음, GPA 계산 불가 등
- **AuthorizationError**: 현재 로그인한 사용자에게 해당 기능에 대한 권한이 없을 경우 발생합니다.
- **ValidationError**: 입력값이 유효하지 않거나 시스템 조건에 맞지 않는 경우 발생합니다.

각 예외는 사용자에게 명확한 오류 메시지를 전달하고, 프로그램이 강제 종료되지 않도록 예외 처리 블록(try-except) 내에서 잡혀 CLI에 출력됩니다.

7.2 정규표현식 검사 항목

※ 정규표현식 패턴은 직접 작성 예정

- AcademicError, AuthorizationError, ValidationError 등

7.2 정규표현식 검사 항목

- 학번: `^20\d{6}$`
- 과목코드: `^[A-Z]{3,5}\d{3,4}$`

8. academic_data.json 파일 구조 및 예시

academic_data.json 은 본 시스템의 핵심 데이터 저장소로, 다음 네 가지 키(students, professors, courses, enrollments)를 포함하는 JSON 구조입니다. 시스템 실행 시 이를 파싱하여 각 객체로 복원하며, 종료 시 변경 사항이 반영된 상태로 다시 저장됩니다.

8.1 데이터 생성

이 프로젝트에 사용한 데이터는 모두 직접 만들었습니다. 각 과목과 교수님들의 정보는 SAINT의 개설 교과목 정보에서 2023년 1학기부터 2025년 여름학기 엑셀 데이터를 다운받아 이를 바탕으로 작성했습니다. 데이터는 학생 300명, 교수 40명, 과목 239개, 수강신청 5400건을 만들었으며, 모두 경제학과에 소속되고, 경제학 전공만 수강한다는 가정 하에서 학생 데이터를 만들었습니다. 학생은 23학번 100명, 24학번 100명, 25학번 100명으로 구성되어 있고, 한 학기에 무조건 18학점을 수강했다는 가정이 있습니다. 그리고 학생은 수강신청을 할 수 있는데, 이는 25년도 여름 계절학기를 수강신청 하는 시점이라는 환경을 가정했습니다. 학생의 이름은 임시로 'student20230001'과 같은 형태로 작성하였고, 비밀번호는 초기값인 0000으로 설정하였습니다. 마지막으로 모든 데이터는 JSON 파일로 저장됩니다.

8.2 데이터 구조 요약 표

Key	Type	설명
students	List[Dict]	학번, 이름, 전공, 학년, 국적, 지도교수 등 포함
professors	List[Dict]	사번, 이름, 학과, 직함 등 포함

courses	List[Dict]	과목코드, 이름, 교수, 학점, 정원, 개설 학기 등 포함
enrollments	List[Dict]	student_id, course_code, year, semester, grade 필드 포함

8.3 샘플 구조 JSON

```
{
  "students": [
    {
      "uid": "20230001",
      "name": "Student20230001",
      "major": "경제학",
      "nationality": "한국",
      "role": "student",
      "semesters_completed": 5,
      "year": 3,
      "advisor": "김숙영",
      "current_courses": [],
      "password": "0000"
    },
    ...
  ],
  "professors": [
    {
      "uid": "P00000",
      "name": "전현배",
      "major": "경제학",
      "nationality": "한국",
      "role": "professor",
      "title": "교수",
      "password": "0000"
    },
    { "courses": [
      {
        "code": "ECN0001",
        "major": "경제학",
        "name": "경제학원론 I",
        "section": 1,
        "professor": "P00000",
        "capacity": 85,
        "schedule": "월,수 10:30~11:45 [GN201]",
        "credit": 3,
        "year": 2023,
```

```

    "semester": 1
  },
  ...
],
"enrollments": [
  {
    "student_id": "20230001",
    "course_code": "ECN0049",
    "year": 2023,
    "semester": 1,
    "grade": "D0"
  },
  ...
]
}

```

본 JSON 구조는 CLI 기반 학사 시스템이 빠르게 데이터를 로딩하고 저장할 수 있도록 최적화되어 있으며, 실제 학사 시스템의 DB 테이블 구조를 모사한 설계입니다.

9. 실행 예시 및 CLI 출력 결과

아래는 주요 기능별 CLI 상호작용 예시입니다. 각 기능은 사용자 유형에 따라 분류되며, 메뉴 선택 후 실제 수행되는 흐름을 보여줍니다.

9.1 로그인 및 사용자 분기

- 프로그램 실행 후 사용자 유형(관리자/교수/학생) 선택 및 로그인

```

C:\Users\XH58\Desktop\대학 과제\3학년 2학기\기초빅데이터프로그래밍 (서경희 교수님)\Project>python project.py
데이터 파일을 로딩 중입니다: academic_data.json
데이터 로딩 완료: 학생 300명, 교수 40명, 과목 239개, 수강신청 5400건
관리자 모드로 실행하시겠습니까? [y/n] n
학번/사번을 입력하세요: 20230001
비밀번호를 입력하세요: 0000
환영합니다, Student20230001님!
[학생 모드]
register  수강신청
drop     수강취소
courses  수강한 과목 조회
grades   성적 조회
info     내 정보 열람
quit     종료
>

```

```

데이터 파일을 로딩 중입니다: academic_data.json
데이터 로딩 완료: 학생 300명, 교수 40명, 과목 239개, 수강신청 5400건
관리자 모드로 실행하시겠습니까? [y/n] n
학번/사번을 입력하세요: P00000
비밀번호를 입력하세요: 0000
환영합니다, 전현배님!
[교수 모드]
list      수강 학생 조회
grade     성적 입력
stats     성적 통계/시각화
info      내 정보 열람
quit      종료
>

```

```

C:\Users\XH58\Desktop\대학 과제\3학년 2학기\기초빅데이터프로그래밍 (서경희 교수님)\Project>python project.py
데이터 파일을 로딩 중입니다: academic_data.json
데이터 로딩 완료: 학생 300명, 교수 40명, 과목 239개, 수강신청 5400건
관리자 모드로 실행하시겠습니까? [y/n] y
[관리자 모드]
1. 학생 입력
2. 학생 삭제
3. 총원 조회
4. 종료
>

```

- 잘못된 비밀번호 대한 예외 메시지 출력

9.2 학생 기능 예시

- 수강신청(register): 과목 리스트 출력 → 과목코드 입력 → 등록 성공 여부 출력
-

```

환영합니다, Student20230001님!
[학생 모드]
register   수강신청
drop       수강취소
courses   수강한 과목 조회
grades     성적 조회
info       내 정보 열람
quit       종료
> register

[개설 과목 목록]

```

번호	코드	과목명	담당	연도	학기	분반	학점
1	EC02001	경제학원론 I	김숙영	2025	하계학기	1	3
2	EC02002	경제학원론 II	김형욱	2025	하계학기	1	3
3	EC02003	경제수리기초	박경욱	2025	하계학기	1	3

```

수강하고자 하는 과목 코드: EC02002
분반(숫자): 1
현재 58명이 대기 중입니다.
수강신청에 성공하였습니다.

```

- 수강취소(drop): 현재 수강중인 과목 선택 후 취소 처리

```
[학생 모드]
register  수강신청
drop      수강취소
courses  수강한 과목 조회
grades   성적 조회
info     내 정보 열람
quit     종료
> drop
[2025년 2학기 수강신청 내역]
  1. ECO2002 경제학원론II
취소할 과목 번호(0: 취소): 1
ECO2002 수강신청이 취소되었습니다.
[학생 모드]
register  수강신청
drop      수강취소
courses  수강한 과목 조회
grades   성적 조회
info     내 정보 열람
quit     종료
```

- 수강조회(courses): 수강중인 과목 목록을 표 형식으로 출력

```
> courses
[학기별 수강 과목 및 성적]

--- 2023년 1학기 (과목 수: 6, 평균 GPA: 2.27) ---
```

과목코드	과목명	성적	GPA
ECN0049	고급미시경제학I	D0	1
ECN0027	계량경제학I	A0	4
ECO2001	경제학원론I	B-	2.7
ECN0017	경제통계학	B+	3.3
ECN0033	국제무역론	D+	1.3
ECN0032	공공경제학	D+	1.3

- 성적조회(grades): 학기별 GPA 출력 + GPA 추이 시각화 그래프

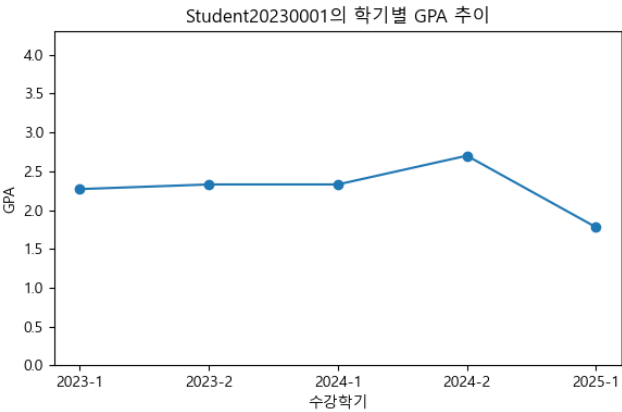
```
> grades
```

[학기별 GPA]

2023년 1학기 GPA: 2.27
 2023년 2학기 GPA: 2.33
 2024년 1학기 GPA: 2.33
 2024년 2학기 GPA: 2.7
 2025년 1학기 GPA: 1.78

[학기별 GPA 표]

수강학기	년도	학기	GPA
1	2023	1	2.27
2	2023	2	2.33
3	2024	1	2.33
4	2024	2	2.7
5	2025	1	1.78



- 내 정보 열람(info): 로그인한 사용자의 정보 출력

```
> info
```

[내 정보]

이름: Student20230001
 학번/사번: 20230001
 전공/학과: 경제학
 국적: KR
 학년: 3
 지도교수: 김숙영

9.3 교수 기능 예시

- 수강생 목록(list): 과목별 수강 학생 테이블 출력

> list
[담당 과목 전체 목록]

번호	코드	과목명	연도	학기	분반	학점
1	ECN0001	경제학원론 I	2023	1	1	3
2	ECN0057	경제학원론 II	2023	2	1	3
3	ECN0080	산업경제학	2023	2	1	3
4	ECN0159	경제학원론 II	2024	2	1	3
5	ECN0178	산업경제학	2024	2	1	3
6	ECN0209	경제학원론 I	2025	1	1	3

과목 번호를 입력하세요 : 2
=== ECN0057 수강 학생 ===

학번	이름
20230002	Student20230002
20230005	Student20230005

과목 번호를 입력하세요 : 2
=== ECN0057 수강 학생 ===

학번	이름
20230002	Student20230002
20230005	Student20230005
20230006	Student20230006
20230007	Student20230007
20230018	Student20230018
20230031	Student20230031
20230038	Student20230038
20230039	Student20230039
20230042	Student20230042
20230043	Student20230043
20230051	Student20230051
20230056	Student20230056
20230057	Student20230057
20230079	Student20230079
20230080	Student20230080
20230081	Student20230081
20230082	Student20230082

- 성적입력(grade): 과목 선택 → 학생 선택 → 성적 등급 입력

> grade

[담당 과목 전체 목록]

번호	코드	과목명	연도	학기	분반	학점
1	ECN0001	경제학원론 I	2023	1	1	3
2	ECN0057	경제학원론 II	2023	2	1	3
3	ECN0080	산업경제학	2023	2	1	3
4	ECN0159	경제학원론 II	2024	2	1	3
5	ECN0178	산업경제학	2024	2	1	3
6	ECN0209	경제학원론 I	2025	1	1	3

과목 번호를 입력하세요 : 2

[수강 학생 목록]

20230002 Student20230002
 20230005 Student20230005
 20230006 Student20230006
 20230007 Student20230007
 20230018 Student20230018
 20230031 Student20230031
 20230038 Student20230038
 20230039 Student20230039
 20230042 Student20230042
 20230043 Student20230043
 20230051 Student20230051
 20230056 Student20230056
 20230057 Student20230057
 20230079 Student20230079
 20230080 Student20230080
 20230081 Student20230081
 20230082 Student20230082
 20230096 Student20230096

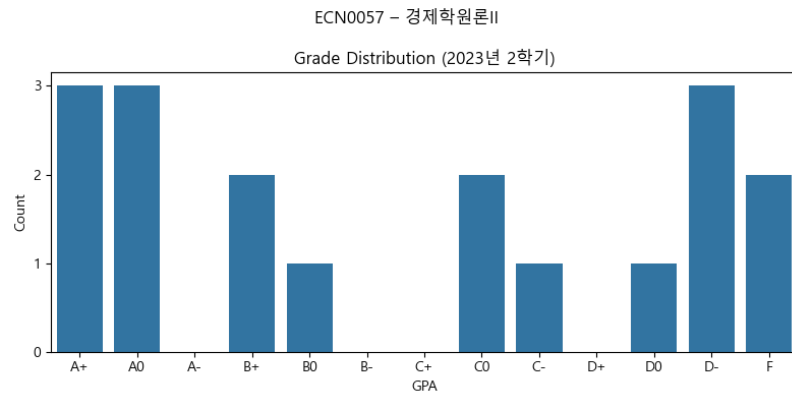
학생 학번 : 20230002

성적 : A+

Student20230002(20230002) → A+

- 성적통계(stats): seaborn 그래프를 통한 성적 분포 시각화 + 통계치 출력

[성적 분포표]	
성적	인원수
A+	3
A0	3
A-	0
B+	2
B0	1
B-	0
C+	0
C0	2
C-	1
D+	0
D0	1
D-	3
F	2



- 내 정보 열람(info): 로그인한 사용자의 정보 출력

```
> info
```

[내 정보]
 이름: 전현배
 학번/사번: P00000
 전공/학과: 경제학
 국적: KR
 직함: 교수

9.4 관리자 기능 예시

- 학생 추가(add): 학번, 전공, 이수학기, 지도교수 등 입력 → 등록 결과 출력

```

[관리자 모드]
1. 학생 입력
2. 학생 삭제
3. 총원 조회
4. 종료
> 1
학번: 20200595
이름: 이석희
전공을 선택하세요:
  1. 경제학
전공 번호: 1
국적: 한국
이수 학기 수: 1
학년(숫자): 1
지도교수 이름: 김서강
입력할 학기 개수(0이면 건너뛸): 1
년도(예: 2024): 2025
학기(1 또는 2): 1
[경제학] 2025년 1학기 개설 전공 과목 목록:
  ECN0209: 경제학원론I
  ECN0210: 경제학원론I
  ECN0211: 경제학원론I
  ECN0212: 경제학원론I
  ECN0213: 경제학원론I
  ECN0214: 경제학원론I
  ECN0215: 경제학원론II
  ECN0216: 경제학원론II
  ECN0217: 경제학원론II
  ECN0218: 경제학원론II
  ECN0219: 경제수리기초
  ECN0220: 경제수리기초
  ECN0221: 경제통계학
  ECN0222: 경제통계학
  ECN0223: 경제통계학
  ECN0224: 미시경제학I
  ECN0225: 미시경제학I
  ECN0226: 미시경제학I
  ECN0227: 미시경제학I

```

```

ECN0228: 거시경제학I
ECN0229: 거시경제학I
ECN0230: 거시경제학I
ECN0231: 계량경제학I
ECN0232: 계량경제학I
ECN0233: 계량경제학I
ECN0234: 동태계획법
ECN0235: 산업경제학
ECN0236: 개발경제학
ECN0237: 국제경제학
ECN0238: 경제성장론
ECN0239: AI금융(캡스톤디자인)
ECN0240: 교통경제학
ECN0241: 경제수리통계학
ECN0242: 게임이론
ECN0243: 경쟁법과산업규제의경제학
ECN0244: 금융정책
ECN0245: 국제금융시장
ECN0246: 경제학특강3
ECN0247: 행동경제학
ECN0248: 금융시장계량분석(캡스톤디자인)
ECN0249: 고급미시경제학I
ECN0250: 고급거시경제학I
ECN0251: 고급계량경제학I
ECN0252: 고급수리통계학
ECN0253: AI와빅데이터를활용한경제정보처리
2025년 1학기 수강 과목 수: 1
수강 과목 코드: ECN0247
성적(공백 입력 시 미입력): A+
등록 완료.
[Time] _admin_add_student took 29.6515 s

```

- 학생 삭제(delete): 삭제할 학번 입력 → 관련 수강기록 포함 삭제 처리

```

[관리자 모드]
1. 학생 입력
2. 학생 삭제
3. 총원 조회
4. 종료
> 2
삭제할 학생의 학번: 20200595
학생 및 관련 수강 이력이 삭제되었습니다.
[Time] admin delete student took 1.3751 s

```

- 전체 통계(show_counts): 등록된 학생, 교수, 과목 수 출력 + 실행 시간 측정

```
[관리자 모드]
1. 학생 입력
2. 학생 삭제
3. 총원 조회
4. 종료
> 3

[총원 현황]
총 학생 수: 300명
총 교수 수: 40명
총 과목 수: 239개
총 수강신청 건수: 5400건
[Time] _admin_show_counts took 0.0012 s
```